TEC | Tecnológico de Costa Rica

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE 4302 — Arquitectura de Computadores II

Proyecto Grupal 2

Framework experimental de modelos de ejecución **Data-Level Parallelism, DLP**

Fecha de asignación:	5 de mayo 2026	Fecha de entrega:	4 de junio 2026
Grupos:	3 personas	Profesores:	Ronald García Fernández Luis Chavarría Zamora

1. Objetivo

Mediante el desarrollo de este proyecto, la persona estudiante aplicará conceptos de paralelismo a nivel de datos (*Data-Level Parallelism*) utilizando arquitecturas vectoriales, extensiones SIMD y procesamiento mediante GPU.

El objetivo principal consiste en diseñar e implementar un problema altamente paralelizable, analizar su descomposición en operaciones vectoriales y evaluar su desempeño utilizando distintos modelos de paralelismo por datos disponibles en arquitecturas modernas.

La persona estudiante deberá comprender las diferencias entre paralelismo a nivel de instrucción, paralelismo a nivel de datos y paralelismo masivo en GPU, así como las implicaciones de estos modelos en el diseño de algoritmos de alto desempeño.

Atributos relacionados: **DI**seño:

- **Definición:** Diseña soluciones creativas para problemas de ingeniería complejos y diseña sistemas, componentes o procesos para satisfacer las necesidades identificadas con la consideración adecuada para la salud y la seguridad públicas, el costo total de la vida, el carbono neto cero, así como las consideraciones de recursos, culturales, sociales y ambientales según sea necesario.
  - **DI1** - Identifica las necesidades y los requerimientos de un problema complejo de ingeniería considerando la salud y la seguridad pública, el costo total de la vida, el carbono neto cero, así como aspectos relacionados con recursos, culturales, sociales y ambientales según sea necesario.
  - **DI2** - Valora alternativas de solución para un problema complejo de ingeniería que cumplan con necesidades específicas, considerando la salud y la seguridad pública, el costo total de la vida, el carbono neto cero, así como aspectos relacionados con recursos, culturales, sociales y ambientales según sea necesario.

- **DI3** - Diseña de forma creativa, la alternativa seleccionada que cumpla con las necesidades específicas para resolver el problema complejo de ingeniería, considerando la salud y la seguridad pública, el costo total de la vida, el carbono neto cero, así como aspectos relacionados con recursos, culturales, sociales y ambientales según sea necesario.
- **DI4** - Valida el diseño final de acuerdo con los requerimientos, la salud y la seguridad pública, el costo total de la vida, el carbono neto cero, así como aspectos relacionados con recursos, culturales, sociales y ambientales según sea necesario.

2. Descripción general

Las arquitecturas modernas incorporan múltiples mecanismos para explotar paralelismo a nivel de datos, permitiendo ejecutar una misma operación sobre múltiples elementos simultáneamente. Este paradigma se implementa mediante diferentes mecanismos arquitectónicos, entre ellos las extensiones SIMD en procesadores modernos, arquitecturas vectoriales y aceleradores masivamente paralelos como las GPU.

A diferencia del paralelismo basado en hilos, el paralelismo a nivel de datos se enfoca en aplicar una misma operación a grandes conjuntos de datos de manera simultánea. Este modelo es ampliamente utilizado en aplicaciones de alto desempeño tales como procesamiento de imágenes, simulaciones científicas, aprendizaje automático y renderizado gráfico.

En este proyecto se desarrollará un framework experimental que permita evaluar distintas estrategias de paralelismo por datos mediante implementaciones en CPU y GPU. La persona estudiante deberá analizar la descomposición del problema, implementar distintas versiones del algoritmo utilizando técnicas vectoriales y paralelismo masivo, y comparar el desempeño obtenido mediante métricas cuantitativas.

3. Especificación

En este proyecto cada grupo de trabajo debe proponer un problema altamente paralelizable mediante paralelismo a nivel de datos y evaluar su comportamiento bajo distintos modelos de ejecución.

1. El framework debe implementar las siguientes características, eligiendo un hardware objetivo (CPU + GPU, CPU SIMD):

   a) Implementación secuencial base (baseline): versión completamente secuencial que servirá como referencia para medir el desempeño de las demás implementaciones.

b) Paralelismo SIMD en CPU: implementación utilizando extensiones SIMD disponibles en la arquitectura del procesador (por ejemplo AVX, AVX2, AVX-512 o NEON dependiendo del hardware). Se permite utilizar (justificando su elección):

- Intrinsics del compilador.
- Bibliotecas vectoriales.

c) Implementación en GPU: ejecución del algoritmo utilizando una arquitectura GPU mediante una API de cómputo paralelo como:

- CUDA
- OpenCL
- OpenACC
- Vulkan Compute
- Metal (en sistemas Apple)

2. El grupo debe aplicar técnicas de descomposición de problemas para identificar regiones paralelizables mediante paralelismo por datos. Debe justificarse formalmente el modelo de descomposición utilizado, por ejemplo:

   - Descomposición por dominio de datos.
   - Descomposición espacial.
   - Paralelismo de operaciones independientes.
   - Paralelismo de flujo de datos.

   Es necesario que se describa el razonamiento de la optimización algorítmica considerando aspectos como: reorganización de estructuras de datos y operaciones para maximizar vectorización y acceso eficiente a memoria.

3. El problema es propuesto por las personas estudiantes. Se recomiendan problemas con alto grado de paralelismo de datos tales como:

   - Procesamiento de imágenes (convoluciones, filtros, transformadas).
   - Simulaciones físicas (sistemas de partículas).
   - Algoritmos de álgebra lineal (multiplicación de matrices).
   - Renderizado o ray tracing.
   - Procesamiento de grandes arreglos numéricos.

4. Para cuantificar la mejora deben proveer al menos las siguientes métricas:

a) Tiempo total de ejecución.

b) Speedup respecto a la versión secuencial.

c) Eficiencia paralela.

d) Escalabilidad en función del tamaño del problema.

5. Se debe utilizar al menos una herramienta de perfilado o medición de desempeño, por ejemplo:

   - `perf` en Linux
   - NVIDIA Nsight Systems o Nsight Compute
   - Intel VTune Profiler
   - Herramientas de profiling del compilador

   Estas herramientas deberán utilizarse para analizar aspectos como:

   - Utilización de unidades vectoriales
   - Ocupación de GPU
   - Ancho de banda de memoria
   - Latencia de memoria
   - Estadísticas de memoria caché
   - Eficiencia de ejecución paralela
   - Análisis de diferencia numérica (caso GPU + CPU)

Es necesario que se justifique que métricas utilizan y el porqué su elección, considerando características del sistema objetivo, no se prentende capturar datos sin razonamiento previo, esto se penalizará con una nota de cero.

3.1. Notas adicionales

1. El desarrollo de este proyecto es grupal máximo 3 integrantes.

2. El proyecto de poder ser ejecutado en hardware real, es decir no se puede presentar análisis o defensa basado en sistemas virtualizados ya que esto limita la visibilidad de herramientas de perfilado.

3. Deben usar Github, este debe ser privado, usen la metodología indicada en la sección 4.

4. El proyecto tendrá dos demostraciones o evaluaciones antes de la entrega final, estos serán los requerimientos mínimos de los avances:

   a) Demostración 1: Jueves 14 de mayo. El grupo de trabajo debe mostrarle al profesor el problema escogido, las razones para fundamentar su selección. Aquí deberá proponer mediante diagramas (con base en la teoría) sobre cómo implementará el paralelismo SIMD en CPU, implementación en GPU, junto que metricas son relevantes para el sistema objetivo.

   b) Demostración 2: Martes 26 de mayo. El grupo de trabajo debe mostrarle al profesor los diagramas de las soluciones esquematizadas preliminares (indicando los cambios más significativos). Ejecución del sistema base. Identificación de las variables más importantes del problema que aumenten su paralelización. Una ejecución *dummy* o trivial de cada esquema DLP, puede ser en partes parciales del problema.

   Estas demostraciones **son obligatorias** pues permiten tener certeza que el producto final llegue con menos fallas, de no realizarse no se procede a la revisión del proyecto.

5. Todo diseño deberá tener al menos 2 propuestas detalladas adecuadamente y comparadas según criterios.

6. Es necesario que diseñe una estrategia de prueba para la validación de la correctitud de su solución (*secuencial* == *paralelo*), cantidad de pruebas, tamaño de los *datasets*, etc.

4. Metodología de trabajo

El proyecto debe seguir los siguientes aspectos de desarrollo, sino, la parte funcional no será calificada y obtendrá nota de cero. El grupo de trabajo deberá seguir los siguientes lineamientos:

1. Los estudiantes pueden usar herramientas de inteligencia artificial u otras fuentes para guía. Debe documentar los procedimientos automatizados y los mismos deben ser validados por usted contra referencias verificadas y confiables [1]. **En este proyecto ustedes será un diseñadores, no un copilotos(as) sin criterio técnico crítico. Esto debe formar parte de los anexos en el documento entregado.**

2. Todo código debe ser documentado de forma interna.

3. Manejo por repositorio:

   a) Utilice una cuenta de repositorio gratuita.

[1] Algunas veces estas herramientas inducen a sesgos y no deben ser tomadas sin cuestionamiento o verificación (como todo en ingeniería).

b) El repositorio debe ser enviado al profesor no más de 5 hábiles después de la entrega de este enunciado. Sino, habrá **penalización** de un punto por hora en la nota final del proyecto después de las 11:59 pm del quinto día hábil.

c) Cree un repositorio con el siguiente nombre: `<user_id1>_<user_id2>_<user_id3>_a2_2026_s1`. El `user_id` estará compuesto por la primera letra del nombre y el apellido. Por ejemplo, para las personas estudiantes Fulanito Pérez, Jane Doe y el nombre del repositorio será: `fperez_jdoe_a2_2026_s1`.

d) El repositorio debe ser mediante el uso de GitHub, debe ser privado, proporcione acceso a `luchazam` (Grupo 01) o `rgarciatec` (Grupo 02). Si no es privado desde el inicio recibirá penalización de 10 puntos.

e) El repositorio de Git contendrá dos ramas principales: `master` y `development`.

f) Inicialmente, la rama de `development` se crea a partir del `master`.

g) Al trabajar en un proyecto, el grupo del trabajo debe crear una nueva rama de trabajo desde `develop` y cuando la función esté lista, la rama debe fusionarse para `develop`. Cualquier corrección o modificación adicional después de `merge` debería requerir que se repita el proceso (es decir, crear la rama desde `develop` y fusionar los cambios más tarde). Una vez que el código de desarrollo esté listo, se fusionará con `master` y se debe crear una `tag`. El proceso se describe en la Figura 1.

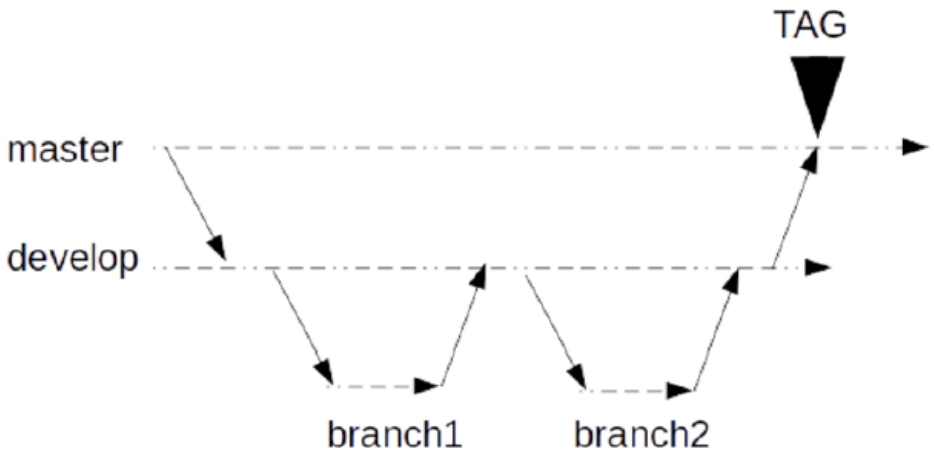


Figura 1: *Git workflow*

Adicionalmente se coloca este enlace recomendado.

h) Solo debe contemplar código, recuerde que es un repositorio de versiones de código, no un Drive.

i) Después de haber realizado algunos proyectos la rama `master` debe verse así:

- `master/`
  - `proyecto_1`
  - `proyecto_2`
  - ...
- ...

Donde cada directorio de `proyecto_x` contiene todos los entregables para cada proyecto.

No es permitido realizar todo el trabajo en un solo commit, es decir, que realice el trabajo de forma local y solo suba el último entregable en el repositorio. Si no, será penalizado en todos los rubros de evaluación de la defensa disminuyendo un escalafón en la calificación de cada rubro. Debe mostrar avance incremental (se revisarán estadísticas).

5. Entregables

Como entregables en este proyecto se evaluará lo siguiente:

- Presentación funcional completa de **30 minutos** (75 %): Cada persona del grupo deberá demostrar en una sesión (previa cita con el profesor) las diferentes pruebas sobre el sistema. El profesor evaluará las pruebas según rúbrica correspondiente, se realizarán preguntas orales de lo implementado (todo debe ser fundamentado, sino habrá penalidad conceptual). En la sesión se harán preguntas relacionadas sobre cualquier etapa del sistema, se contrastará contra la . El día de la presentación, el grupo de trabajo cargará el último commit antes de la primera revisión.

- Artículo tipo *paper* en formato PDF (25 %): El artículo deberá tener una extensión no mayor a 4 páginas y debe seguir el formato IEEE Transactions. Se recomienda el uso de LaTeX para su elaboración, sin embargo, no es obligatorio; la persona estudiante puede utilizar cualquier herramienta de edición siempre que el documento final se entregue en formato PDF y respete el formato IEEE indicado. Este documento en formato *paper* deberá contar con las siguientes secciones:

1. Motivación y contexto del problema planteado.

2. Identificación de necesidades y requerimientos (funcionales y no funcionales) considerando eficiencia energética, uso eficiente de recursos, rendimiento (throughput), y balance entre complejidad y funcionalidad.

3. Valoración de alternativas de solución. En estas alternativas evaluadas deben primar aspectos técnicos mucho más allá de aspectos de disponibilidad.

4. Diseño creativo de la solución: descripción detallada de la solución, diagramas de bloques del sistema completo y de módulos principales, descripción del sistema.

5. Validación del diseño: Resultados claves del sistema. Procure que los resultados tengan sentido.

6. Análisis de resultados: Comparación cualitativa y cuantitativa de rendimiento del sistema.

7. Conclusiones y recomendaciones.

8. Referencias bibliográficas.

Si tienen dudas puede escribir al profesor al correo electrónico de Luis Chavarría, correo electrónico de Ronald García. **Los documentos serán sometidos a control de plagios para eliminar cualquier intento de plagio con trabajos de semestres anteriores, actual o copias textuales, tendrán nota de cero los casos detectados**. La entrega se debe realizar por medio de TEC-Digital en la pestaña de evaluación a más tardar las 11:59 pm, no se aceptan entregas extratemporales. Los documentos serán sometidos a control de plagios. **Se prohíbe el uso de referencias hacia sitios no confiables.**